



2026

Seguridad en Base de Datos Relacionales

- Amenazas, controles y buenas practicas

CATEDRA: BASE DE DATOS I

ALUMNO: DAMIAN EMILIANO CASTRO

CARRERA: LICENCIATURA EN SISTEMAS DE INFORMACION

AÑO: 3° AÑO

PROFESORES

PROF. TIT. ING. EMILIO REARTE

PROF. AY. 1RA. ING. JOSÉ L. ANDRADA

ÍNDICE

1. Introducción: ¿Por qué importa proteger una base de datos?
2. ¿Quién quiere atacar tu base de datos y cómo lo hace?
 - 2.1. SQL Injection
 - 2.2. Acceso no autorizado
 - 2.3. Escalada de privilegios
 - 2.4. Ataques internos
3. Quién puede ver qué: control de acceso y gestión de usuarios
 - 3.1. Principio de mínimo privilegio
 - 3.2. Usuarios y roles en MySQL
4. El arte de hacer ilegibles tus datos: cifrado
 - 4.1. Cifrado en reposo
 - 4.2. Cifrado en tránsito
 - 4.3. Hashing de contraseñas
5. El registro que todo lo ve: auditoría y logs
6. Buenas prácticas: guía para una base de datos segura
7. Una base de datos segura no es un lujo, es una necesidad
8. Referencias bibliográficas

1. INTRODUCCIÓN: ¿POR QUÉ IMPORTA PROTEGER UNA BASE DE DATOS?

Para comprender la magnitud de la seguridad informática en las organizaciones, primero debemos definir el núcleo que resguarda su información. De acuerdo a la teoría de sistemas, una base de datos (BD) es una estructura orientada al almacenamiento en un sistema de información, cuyo propósito es garantizar dos principios esenciales: la integración y la independencia de los datos.

En los sistemas modernos, el rol de la BD es vital. Los datos han dejado de ser archivos aislados para convertirse en un recurso integrador compartido por toda la organización. La base de datos actúa como la "memoria del sistema", permitiendo acumular experiencia, evaluar el pasado y retroalimentarse para tomar decisiones. Precisamente porque centraliza este recurso, el objetivo máximo de un Sistema de Base de Datos es proporcionar independencia, integración, privacidad e integridad física y lógica.

Sin embargo, esta centralización también representa su mayor vulnerabilidad. Si el núcleo es comprometido, colapsa toda la operatividad de la organización. Casos como la filtración de Yahoo en 2016 —donde fueron expuestos los datos de más de 3.000 millones de cuentas— o la brecha de Facebook en 2021 —que afectó a 533 millones de usuarios— demuestran que ningún sistema está exento de riesgos, y que las consecuencias van desde pérdidas económicas millonarias hasta daños irreparables a la reputación institucional.

Frente a este panorama, la seguridad en bases de datos relacionales no puede tratarse como un aspecto secundario del diseño, sino como una condición fundamental desde el primer momento. La presente monografía aborda las principales amenazas que enfrentan las bases de datos, los mecanismos de control de acceso y cifrado disponibles, el rol de la auditoría como herramienta de trazabilidad, y un conjunto de buenas prácticas orientadas a reducir la exposición al riesgo.

2. ¿QUIÉN QUIERE ATACAR TU BASE DE DATOS Y CÓMO LO HACE?

Las bases de datos relacionales son uno de los objetivos más codiciados por actores maliciosos, ya que concentran información crítica de organizaciones enteras en un único punto. Las amenazas que enfrentan no provienen únicamente del exterior: pueden originarse dentro de la propia organización, a través de configuraciones incorrectas, malas prácticas o acciones deliberadas. A continuación, se describen los vectores de ataque más frecuentes.

2.1. SQL INJECTION

El SQL Injection es, según OWASP, uno de los ataques más comunes y peligrosos sobre sistemas que utilizan bases de datos. Consiste en la inserción de código SQL malicioso dentro de campos de entrada de una aplicación, con el objetivo de manipular las consultas que esta envía a la base de datos.

Para ilustrarlo, consideremos un formulario de inicio de sesión que construye la siguiente consulta:

```
SELECT * FROM usuarios WHERE usuario = 'admin' AND password = '1234';
```

Si el sistema no valida correctamente la entrada del usuario, un atacante puede ingresar como nombre de usuario el valor **' OR '1'='1'**, transformando la consulta en:

```
SELECT * FROM usuarios WHERE usuario = " OR '1'='1' AND password = ";
```

Como la condición **'1'='1'** siempre es verdadera, la consulta devuelve todos los registros de la tabla, otorgando acceso sin necesidad de credenciales válidas.

La forma más efectiva de prevenirlo es mediante el uso de sentencias preparadas (*prepared statements*), que separan el código SQL de los datos ingresados por el usuario, eliminando la posibilidad de que una entrada maliciosa altere la lógica de la consulta.

2.2. ACCESO NO AUTORIZADO

El acceso no autorizado ocurre cuando un usuario —sea externo o interno— accede a datos para los cuales no tiene permiso. Este tipo de amenaza suele originarse en dos causas principales: la asignación excesiva de privilegios y el uso de credenciales débiles o reutilizadas.

En muchas organizaciones, por comodidad o descuido, se otorga a los usuarios más permisos de los estrictamente necesarios. Un empleado del área de ventas que tiene acceso de lectura y escritura sobre tablas de recursos humanos o finanzas representa un riesgo latente, aunque su intención no sea maliciosa. Del mismo modo, el uso de contraseñas simples como 123456 o admin facilita enormemente el trabajo de un atacante que intente acceder al sistema mediante fuerza bruta o diccionario.

La mitigación de este riesgo requiere aplicar el principio de mínimo privilegio —desarrollado en la sección siguiente— y establecer políticas de contraseñas robustas que incluyan longitud mínima, combinación de caracteres y renovación periódica.

2.3. ESCALADA DE PRIVILEGIOS

La escalada de privilegios es un ataque en el que un usuario con permisos limitados logra obtener un nivel de acceso superior al que le fue asignado, llegando en algunos casos a obtener control total sobre la base de datos.

Este tipo de ataque puede producirse de dos formas. En la escalada vertical, el usuario asciende en el nivel de privilegios, por ejemplo de usuario estándar a administrador. En la escalada horizontal, el usuario accede a recursos de otro usuario con el mismo nivel de permisos, pero diferente ámbito de datos.

Las causas más habituales incluyen vulnerabilidades en el motor de base de datos que no han sido parcheadas, configuraciones incorrectas en la asignación de roles, o el aprovechamiento de funciones del sistema con permisos excesivos. Un caso típico en MySQL es el uso indebido del rol **SUPER**, que otorga capacidades administrativas amplias y no debería asignarse a usuarios operativos.

Mantener el motor de base de datos actualizado, revisar periódicamente la asignación de roles y evitar el uso de cuentas con privilegios elevados para tareas cotidianas son las principales medidas de prevención.

2.4. ATAQUES INTERNOS

No todas las amenazas provienen del exterior. Los ataques internos, protagonizados por empleados, ex empleados o colaboradores con acceso legítimo al sistema, representan uno de los riesgos más difíciles de detectar y prevenir.

Un empleado con acceso a tablas de datos sensibles —como información personal de clientes, registros médicos o datos financieros— puede, de forma intencional o accidental, filtrar, modificar o eliminar información crítica. La peligrosidad de este vector radica en que el actor ya cuenta con credenciales válidas, lo que hace que su actividad no dispare las alertas habituales de un ataque externo.

Las medidas más eficaces para mitigar este riesgo incluyen la implementación de logs de auditoría que registren toda actividad sobre datos sensibles, la revocación inmediata de accesos al momento de la desvinculación de un usuario, y la segmentación del acceso a datos según el rol y la necesidad real de cada empleado.

3. QUIÉN PUEDE VER QUÉ: CONTROL DE ACCESO Y GESTIÓN DE USUARIOS

El control de acceso es el conjunto de mecanismos que determina quién puede interactuar con la base de datos, de qué manera y sobre qué datos específicos. No basta con proteger el perímetro externo del sistema: si una vez dentro cualquier usuario puede acceder a cualquier tabla, leer cualquier dato o modificar cualquier registro, la seguridad de la base de datos es prácticamente inexistente. El control de acceso es, entonces, la primera línea de defensa interna.

3.1. PRINCIPIO DE MÍNIMO PRIVILEGIO

El principio de mínimo privilegio establece que cada usuario debe contar únicamente con los permisos estrictamente necesarios para cumplir su función, y ninguno más. Este principio es una de las bases del diseño seguro en cualquier sistema de información, y su aplicación en bases de datos relacionales es especialmente crítica dado el volumen y la sensibilidad de los datos que estas concentran.

Cuando no se aplica este principio, las consecuencias pueden ser graves. Un usuario con permisos de escritura sobre tablas que solo debería leer puede, accidental o intencionalmente, modificar o eliminar registros. Un desarrollador con acceso directo a producción puede alterar datos reales durante una prueba. Un empleado desvinculado cuya cuenta no fue revocada sigue teniendo acceso total al sistema. En todos estos casos, el problema no es técnico sino de diseño: los permisos fueron asignados sin criterio ni revisión.

Aplicar el principio de mínimo privilegio implica definir con precisión qué puede hacer cada rol dentro del sistema antes de crear usuarios, y revisarlo periódicamente a medida que cambian las necesidades de la organización.

3.2. USUARIOS Y ROLES EN MYSQL

MySQL ofrece un sistema de gestión de usuarios y privilegios que permite implementar el principio de mínimo privilegio de forma granular. Las tres sentencias fundamentales son **CREATE USER** para crear cuentas, **GRANT** para otorgar permisos y **REVOKE** para retirarlos.

A continuación se presenta un ejemplo práctico con tres roles típicos de un sistema de gestión: administrador, empleado y auditor.

Creación de usuarios:

```
62 • CREATE USER 'admin_bd'@'localhost' IDENTIFIED BY 'ClaveSegura#2025';
63 • CREATE USER 'empleado_01'@'localhost' IDENTIFIED BY 'Emp#Clave2025';
64 • CREATE USER 'auditor_01'@'localhost' IDENTIFIED BY 'Aud#Clave2025';
```

Asignación de permisos según rol:

```
66 -- Administrador: control total sobre la base de datos
67 • GRANT ALL PRIVILEGES ON sistema_gestion.* TO 'admin_bd'@'localhost';
68
69 -- Empleado: solo puede consultar e insertar en las tablas de su área
70 • GRANT SELECT, INSERT ON sistema_gestion.clientes TO 'empleado_01'@'localhost';
71 • GRANT SELECT, INSERT ON sistema_gestion.pedidos TO 'empleado_01'@'localhost';
72
73 -- Auditor: solo puede consultar, sin modificar nada
74 • GRANT SELECT ON sistema_gestion.* TO 'auditor_01'@'localhost';
```

Verificación de permisos asignados:

```
76 • SHOW GRANTS FOR 'empleado_01'@'localhost';
```

Justificación: Este esquema refleja directamente el principio de mínimo privilegio: el auditor no puede modificar nada, el empleado solo accede a las tablas que necesita, y el administrador es la única cuenta con control total. En un entorno real, la cantidad de roles y la granularidad de los permisos crecen según la complejidad del sistema, pero la lógica de diseño se mantiene igual.

4. EL ARTE DE HACER ILEGIBLES TUS DATOS: CIFRADO

El control de acceso y la gestión de usuarios son medidas esenciales, pero no suficientes por sí solas. ¿Qué ocurre si un atacante logra acceder directamente a los archivos físicos del servidor, copiar el disco, o interceptar el tráfico de red entre la aplicación y la base de datos? En ese escenario, los permisos ya no sirven de nada. El cifrado es la respuesta a esa pregunta: su función es garantizar que, incluso si los datos son obtenidos de forma ilegítima, resulten completamente inútiles para quien los posea sin la clave correcta.

4.1. CIFRADO EN REPOSO

El cifrado en reposo protege los datos almacenados en disco. Su objetivo es que si alguien obtiene acceso físico al servidor o a los archivos de la base de datos, no pueda leer su contenido sin la clave de descifrado correspondiente.

El algoritmo más utilizado para este propósito es **AES** (*Advanced Encryption Standard*), considerado el estándar de facto en cifrado simétrico. En el contexto de bases de datos, su implementación más práctica es a través de **TDE** (*Transparent Data Encryption*), una tecnología que cifra y descifra los datos de forma automática y transparente para la aplicación: los datos se escriben cifrados en disco y se descifran al momento de ser leídos por el motor, sin que la aplicación deba hacer nada especial.

MySQL soporta TDE a nivel de tablespace desde la versión 5.7, lo que permite cifrar los archivos .ibd donde se almacenan las tablas InnoDB sin modificar ni una línea del código de la aplicación. Esto lo convierte en una de las medidas de seguridad más fáciles de implementar con mayor impacto.

4.2. CIFRADO EN TRÁNSITO

El cifrado en tránsito protege los datos mientras viajan entre la aplicación y la base de datos a través de la red. Sin este tipo de cifrado, cualquier actor que tenga acceso a la red —ya sea mediante un ataque de intermediario (*man-in-the-middle*) o simplemente monitoreando el tráfico— puede capturar credenciales, consultas y resultados en texto plano.

El protocolo estándar para cifrar comunicaciones en red es **TLS** (*Transport Layer Security*), sucesor de SSL. MySQL permite configurar conexiones TLS de forma que el servidor exija certificados válidos antes de aceptar cualquier conexión, garantizando tanto la confidencialidad como la autenticidad del canal.

Una forma sencilla de verificar si una conexión a MySQL está utilizando cifrado es ejecutar:

```
--  
59 • SHOW STATUS LIKE 'Ssl_cipher';
```


4.3. HASHING DE CONTRASEÑAS

Almacenar contraseñas en texto plano en una base de datos es uno de los errores más graves que puede cometer un desarrollador. Si la base de datos es comprometida, todas las contraseñas quedan expuestas de forma inmediata y definitiva. La solución no es el cifrado —que es reversible— sino el hashing, un proceso unidireccional que transforma la contraseña en una cadena irreversible.

Para ilustrar la diferencia, la siguiente tabla muestra cómo se vería un campo password en la tabla usuarios según cómo se haya almacenado:

COMPARATIVA DE ESTÁNDARES DE SEGURIDAD			
ESTÁNDAR DE RIESGO: TEXTO PLANO		ESTÁNDAR MODERNO: BCRYPT	
usuario	password (mal)	usuario	password (bien – bcrypt)
juan.perez	clave1234	juan.perez	\$2b\$12\$KIX8H3z3Xm5aG4v...
maria.gomez	maria2025	maria.gomez	\$2b\$12\$9mNpQ7w3X...v...
admin	admin123	admin	\$2b\$12\$Lz3Rv2k3Xm5aG4v...
VULNERABLE A FILTRACIONES: Las contraseñas son visibles		HASH FUERTE CON SALT: Encriptación segura con Bcrypt	

Explicación: Comparativa entre almacenamiento de contraseñas en **texto plano** y con **hashing bcrypt**.

Los algoritmos recomendados para este propósito son **bcrypt** y SHA-256. bcrypt es especialmente adecuado para contraseñas porque incluye un factor de costo configurable que hace que el proceso de hashing sea deliberadamente lento, dificultando los ataques de fuerza bruta. SHA-256, por su parte, es más rápido y se usa comúnmente para verificar integridad de archivos, aunque para contraseñas se prefiere bcrypt o su variante Argon2.

Un punto importante: el **hashing** debe combinarse siempre con un **salt** —un valor aleatorio único por usuario que se agrega a la contraseña antes de hashearla— para evitar que dos usuarios con la misma contraseña produzcan el mismo hash, y para inutilizar los ataques por tablas precomputadas (*rainbow tables*). bcrypt incorpora el salt de forma automática, lo que lo convierte en la opción más práctica y segura para almacenar contraseñas en una base de datos.

5. EL REGISTRO QUE TODO LO VE: AUDITORÍA Y LOGS

Una base de datos puede tener control de acceso bien configurado, cifrado activo y contraseñas hasheadas, y aun así sufrir un incidente de seguridad. La pregunta entonces no es solo cómo prevenir ataques, sino también cómo detectarlos y reconstruir lo que ocurrió una vez que suceden. Para eso existe la auditoría de bases de datos.

Un log de auditoría es un registro cronológico de eventos relevantes dentro del sistema: quién accedió, qué consultó, qué modificó y cuándo. Su función es doble: por un lado permite detectar comportamientos anómalos en tiempo real o cercano a él; por otro, sirve como evidencia forense en caso de que ocurra un incidente de seguridad.

Los eventos que conviene registrar no son todos por igual. Registrar absolutamente cada consulta generaría un volumen de datos imposible de analizar, por lo que se recomienda enfocarse en los de mayor sensibilidad: intentos de inicio de sesión exitosos y fallidos, cambios en la estructura de tablas (**ALTER TABLE**, **DROP TABLE**), modificaciones sobre datos críticos (**UPDATE**, **DELETE**), creación o eliminación de usuarios y permisos, y accesos a tablas que contengan datos personales o financieros.

En cuanto a su implementación, MySQL ofrece dos mecanismos principales. El *General Query Log* registra todas las consultas recibidas por el servidor, lo que lo hace útil para auditorías detalladas aunque con un costo en rendimiento. El *Binary Log*, por su parte, registra todos los cambios realizados sobre los datos y es fundamental tanto para auditoría como para recuperación ante desastres. Para activar el General Query Log:

```
61 • SET GLOBAL general_log = 'ON';  
62 • SET GLOBAL general_log_file = '/var/log/mysql/general.log';
```

Y para verificar el estado del Binary Log:

```
64 • SHOW VARIABLES LIKE 'log_bin';
```

Finalmente, la auditoría no es solo una buena práctica técnica: en muchos contextos es un requisito legal. El Reglamento General de Protección de Datos de la Unión Europea (GDPR) exige que las organizaciones puedan demostrar el cumplimiento de sus políticas de privacidad, lo que implica mantener registros de acceso a datos personales. En Argentina, la Ley 25.326 de Protección de los Datos Personales establece obligaciones equivalentes para quienes administren bases de datos con información de ciudadanos, incluyendo la obligación de garantizar la seguridad e integridad de los datos y notificar cualquier incidente. El incumplimiento de estas normativas no solo expone a la organización a sanciones económicas, sino también a daños reputacionales difíciles de revertir.

6. BUENAS PRÁCTICAS: GUÍA PARA UNA BASE DE DATOS SEGURA

Las secciones anteriores abordaron cada amenaza y mecanismo de defensa de forma individual. Sin embargo, la seguridad de una base de datos no se logra aplicando una sola medida, sino combinando varias de forma coherente y sostenida en el tiempo. Esta sección reúne, en forma de guía práctica, las acciones concretas que todo administrador o desarrollador debería implementar para reducir significativamente la exposición al riesgo.

*** Aplicar el principio de mínimo privilegio a todos los usuarios**

Cada usuario debe tener acceso únicamente a los datos y operaciones que necesita para cumplir su función. Ningún permiso adicional debe otorgarse "por las dudas".

*** Usar contraseñas fuertes y almacenarlas siempre con hashing**

Las contraseñas de acceso a la base de datos deben ser complejas y únicas. Las contraseñas de los usuarios del sistema nunca deben guardarse en texto plano: siempre con bcrypt u otro algoritmo de hashing robusto con salt.

*** Activar cifrado en reposo y en tránsito**

Los datos almacenados en disco deben estar cifrados mediante TDE o AES. Las conexiones entre la aplicación y la base de datos deben usar TLS para evitar la interceptación del tráfico.

*** Mantener el motor de base de datos actualizado**

Las actualizaciones de MySQL y otros motores incluyen parches de seguridad que corrigen vulnerabilidades conocidas. Un motor desactualizado es un vector de ataque evitable.

*** Realizar backups periódicos y almacenarlos cifrados**

Los respaldos son la última línea de defensa ante pérdida o corrupción de datos. Deben realizarse con frecuencia definida, almacenarse en ubicaciones separadas del servidor principal y estar cifrados para evitar que una copia de seguridad se convierta en una brecha de seguridad.

*** Activar y revisar los logs de auditoría regularmente**

De nada sirve tener logs si nadie los revisa. Deben establecerse rutinas de revisión periódica y, de ser posible, configurar alertas automáticas ante eventos anómalos como múltiples intentos de login fallidos.

*** Usar sentencias preparadas para evitar SQL Injection**

Toda consulta que incluya datos ingresados por el usuario debe construirse mediante *prepared statements*. Esta medida elimina prácticamente por completo el riesgo de SQL Injection.

*** Limitar el acceso a la BD desde redes externas mediante firewall**

La base de datos no debería ser accesible directamente desde internet. El acceso debe restringirse a las direcciones IP de los servidores de aplicación autorizados, reduciendo drásticamente la superficie de ataque.

*** Revisar y revocar permisos de usuarios inactivos o desvinculados**

Los accesos de empleados que cambian de rol o se desvinculan de la organización deben revocarse de forma inmediata. Una cuenta activa sin titular es una puerta abierta.

*** Documentar la política de seguridad de la BD**

Todas las decisiones de seguridad deben estar documentadas: qué roles existen, qué permisos tienen, qué se audita y con qué frecuencia se revisan los accesos. La seguridad sin documentación depende de las personas, no del sistema.

7. UNA BASE DE DATOS SEGURA NO ES UN LUJO, ES UNA NECESIDAD

A lo largo de esta monografía se recorrieron los principales aspectos que hacen a la seguridad de una base de datos relacional: las amenazas más frecuentes, los mecanismos de control de acceso, el cifrado como última barrera ante el robo físico de datos, la auditoría como herramienta de trazabilidad, y un conjunto de buenas prácticas que sintetizan todo lo anterior en acciones concretas. Cada uno de estos elementos, por separado, aporta una capa de protección. Juntos, conforman una postura de seguridad real.

Pero más allá de los conceptos técnicos, hay una reflexión que vale la pena hacer: la seguridad de una base de datos no es responsabilidad exclusiva de un área de sistemas o de un especialista en ciberseguridad. Es, en gran medida, responsabilidad del desarrollador que diseña el esquema de permisos, del administrador que configura el servidor, y del equipo que decide qué datos se guardan y cómo. Cada decisión de diseño tiene implicancias de seguridad, aunque en el momento no lo parezca.

Uno de los errores más comunes en el desarrollo de sistemas es tratar la seguridad como algo que se agrega al final, cuando el sistema ya está funcionando. Se construye toda la lógica, se diseñan las tablas, se implementan las funcionalidades, y recién entonces alguien pregunta: "¿y los permisos?". Para ese momento, corregir la arquitectura es costoso, incómodo y, en muchos casos, incompleto. La seguridad no es un módulo que se instala al terminar: es una decisión que se toma desde la primera línea del diseño.

Los datos que una organización almacena en su base de datos no le pertenecen solo a ella. Detrás de cada registro hay una persona, una transacción, una historia. Proteger esos datos no es un lujo técnico ni una exigencia burocrática: es una responsabilidad concreta hacia quienes confiaron su información al sistema. Y esa responsabilidad empieza mucho antes de que el sistema salga a producción.

8. REFERENCIAS BIBLIOGRAFICAS

MySQL. (2024). MySQL 8.0 Reference Manual – Security. Oracle Corporation.
<https://dev.mysql.com/doc/refman/8.0/en/security.html>

OWASP Foundation. (2021). OWASP Top Ten – A03:2021 Injection.
<https://owasp.org/Top10/>

Argentina. Ley 25.326 de Protección de los Datos Personales (2000). Boletín Oficial de la República Argentina.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7.a ed.). McGraw-Hill Education.

IBM. (2023). Data Security Best Practices. IBM Documentation.
<https://www.ibm.com/docs/en/db2>

Handbook of Applied Cryptography. CRC Press. -> <https://cacr.uwaterloo.ca/hac/>